

quizgen

Automated Test Assembly from a Question Bank

Selecting blueprint-compliant exams from a pool of tagged questions via optimisation

Master's Thesis — Implementation Overview

Jiniku Hayashi · Advisor: Prof. Michael Kohlhase

KWARC Group · FAU Erlangen-Nürnberg · June 24, 2026

- 1 What the tool does (and does *not*) do
- 2 Core idea: pool, blueprint, versions
- 3 The selection pipeline
- 4 Data model & the blueprint
- 5 Resolving proportions to exact counts
- 6 How questions are picked (greedy / MIP)
- 7 Deduplication & validation
- 8 Command-line interface
- 9 Provenance of techniques
- 10 Summary

The task

Given an existing **question bank** (a JSON list of tagged questions), *select* the subset(s) that form a professor's exam, exactly matching a **blueprint**.

Deliberately narrow scope:

- No question *generation*. No document *ingestion*.
- Reasons only over question *metadata* and the requested *constraints*.
- No LLM, no internet — runs fully offline.

Why this is the right framing

Selection becomes a **well-posed combinatorial optimisation problem** with an **exact, auditable** solution — unlike prompting a model to “write the exam”, which couples creation with constraint satisfaction and gives neither guarantees nor traceability.

Pool

Your question bank — the input JSON. Each item carries **chapter, difficulty, type, Bloom level**.

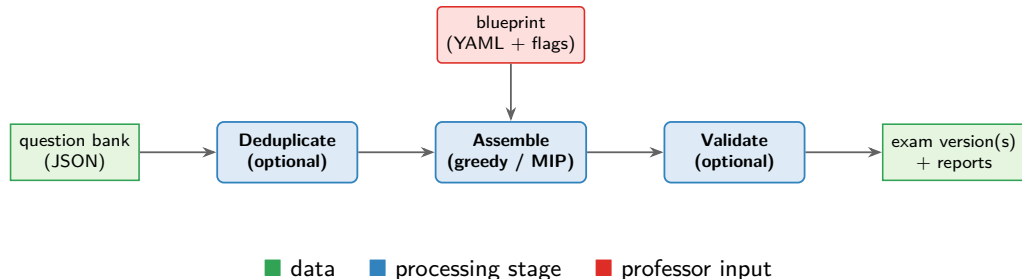
Blueprint

The exam specification: **how many, per chapter, difficulty mix, allowed types, #versions**.

Version

One assembled exam paper. Several at once — **provably sharing no question**.

Example: 180-question pool $\Rightarrow$ a 20-question exam, 10/6/4 across three chapters, an 8/8/4 easy/medium/hard mix, in **two disjoint versions**.



The blueprint **drives** assembly; dedup and validation are opt-in. Each stage maps to one module (schema, blueprint, dedup/similarity, assemble, validate).

Every artifact is a **Pydantic v2** model; the JSON Schema is *exported* from it (so the two cannot drift).

```
class Question(BaseModel):
    id: str                # uuid4 hex[:12]
    chapter: int           # >= 1
    qtype: QuestionType   # mcq / true_false / short_answer / cloze
    bloom_level: BloomLevel # remember .. create (revised Bloom, 2001)
    difficulty: Difficulty # easy / medium / hard
    stem: str; options: list[str] | None; answer: str
    explanation: str       # >= 10 chars
    source_ref: str        # provenance (free-form)
```

Field	Drives
chapter	per-chapter quotas
difficulty	the easy/medium/hard mix
qtype	allowed-type filtering
bloom_level	the quality objective
stem/answer/options	near-duplicate detection

A small, professor-facing YAML file that translates pedagogical intent into measurable selection constraints:

```
title: "AI-1 Midterm Exam"
total_questions: 20
chapters: { 1: "50%", 2: "30%", 3: "20%" }
difficulty_mix: { easy: 0.4, medium: 0.4, hard: 0.2 }
allowed_qtypes: [mcq, true_false, short_answer, cloze]
versions: 2
selector: mip           # greedy | mip
dedup_similarity: 0.85
```

- Amounts may be **exact counts** (10), **fractions** (0.4), or **percentages** ("50%").
- `versions: 2` $\Rightarrow$ two papers that **share no question**.

An exam cannot contain half a question, so `Blueprint.resolve()` turns every proportional request into an integer quota *before* assembly.

Chapter	Share	Count
Ch. 1	50%	$0.50 \cdot 20 = 10$
Ch. 2	30%	$0.30 \cdot 20 = 6$
Ch. 3	20%	$0.20 \cdot 20 = 4$
Total	100%	20

Difficulty	Share	Count
Easy	0.4	$0.4 \cdot 20 = 8$
Medium	0.4	$0.4 \cdot 20 = 8$
Hard	0.2	$0.2 \cdot 20 = 4$
Total	1.0	20

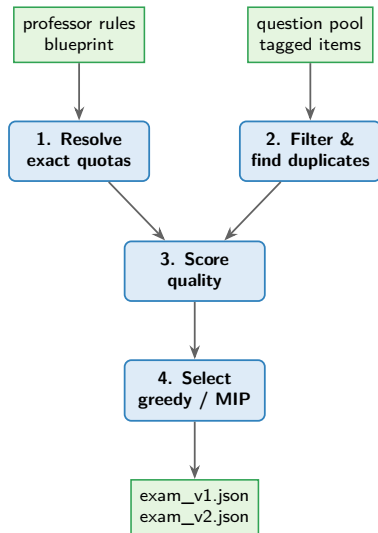
When shares don't divide evenly: give each category its whole part, then hand the leftover units to the **largest fractional remainders**.

$$\text{raw}_k = w_k N, \quad \text{floor}_k = \lfloor \text{raw}_k \rfloor, \quad r = N - \sum_k \text{floor}_k$$

Worked example: $N = 21$, shares 50/30/20 %

$\text{raw} = 10.5, 6.3, 4.2 \rightarrow \text{floors} = 10, 6, 4$ (one left) $\rightarrow$ largest remainder 0.5 is Ch. 1 $\rightarrow$ **11, 6, 4** (sums to 21).

```
def largest_remainder(weights, total):
    raw = {k: weights[k] * total for k in weights}
    counts = {k: int(raw[k]) for k in weights}
    remaining = total - sum(counts.values())
    order = sorted(weights, key=lambda k: raw[k] - counts[k], reverse=True)
    for k in order[:remaining]:
        counts[k] += 1
    return counts
```



- 1 Resolve** the blueprint into a normalised demand vector.
- 2 Filter** out disallowed types; detect near-duplicate pairs.
- 3 Score** each candidate's quality (a selection preference).
- 4 Select** complete versions — greedy or MIP.

Variants that differ by a phrase (“... (*variant 1*)” vs “... (*variant 2*)”) test the same content — they should not co-occur.

Default backend — token-Jaccard (dependency-free):

$$\text{sim}(q_1, q_2) = \frac{|T_1 \cap T_2|}{|T_1 \cup T_2|} \in [0, 1]$$

T_i = lowercase word tokens from stem + answer + options.

Example

“Define the term uninformed search” vs the same + “variant”: 5 shared / 6 total = $5/6 \approx 0.833$. At threshold **0.85** this pair is *not* blocked; ≥ 0.85 adds it to the duplicate set D .

Optional backend: sentence-transformers embedding cosine — catches paraphrases that share little surface vocabulary.

A lightweight *preference* weight (not a correctness score), used only to break ties among feasible selections:

$$w_i = 1.0 + b_{\text{expl}} + b_{\text{options}} + b_{\text{bloom}} + b_{\text{difficulty}}$$

Component	Rationale
Base 1.0	every valid question has minimal value
Explanation bonus	longer explanations → auditability / feedback
Options bonus	MCQ with exactly four options
Bloom bonus	analyze/evaluate/create (higher order)
Difficulty bonus	small bonus for hard items

e.g. hard MCQ, Bloom evaluate, 4 options, long explanation: $1.0 + 0.30 + 0.20 + 0.20 + 0.10 = \mathbf{1.80}$.

Builds one version, one chapter at a time; reproducible shuffle; difficulty-aware first pass, then fill; blocks near-duplicates after each pick.

```
For each version:
```

```
  For each required chapter:
```

```
    collect candidates; drop used / near-duplicate-blocked; shuffle (seeded)
```

```
    pass 1: choose candidates that still help the difficulty quota
```

```
    pass 2: fill remaining chapter slots with any candidate
```

```
    mark chosen as used; block their near-duplicates
```

Strengths

- Very fast, easy to inspect
- Honours exact chapter counts

Limitations

- No global optimality
- Later versions get leftovers; difficulty only best-effort

Models the **whole** assembly at once — all questions, versions, quotas, exclusions, weights — as one 0–1 (Binary) Integer Program.

Decision variables

$$x_{iv} \in \{0, 1\} \quad \forall i \in \{0, \dots, n-1\}, \forall v \in V$$

($x_{iv} = 1$ iff question i is placed in version v).

Objective — maximise total selected quality

$$\max Z = \sum_{v \in V} \sum_{i=0}^{n-1} w_i x_{iv}$$

With 180 questions $\times$ 2 versions: 360 binary switches; the solver turns exactly the right ones on.

exact chapter counts: $\sum_{i \in Q_c} x_{iv} = n_c \quad \forall c \in C, \forall v \in V$

exact difficulty counts: $\sum_{i \in Q_l} x_{iv} = m_l \quad \forall l \in L, \forall v \in V$

disjoint versions: $\sum_{v \in V} x_{iv} \leq 1 \quad \forall i$

no near-duplicate pair: $\sum_{v \in V} x_{iv} + \sum_{v \in V} x_{jv} \leq 1 \quad \forall (i, j) \in D$

Solved with **PuLP + CBC** via branch-and-bound: an LP relaxation gives upper bounds, fractional variables are branched to 0/1, and dominated branches are pruned — avoiding enumeration of all $2^{|V|}$ settings.

Sometimes the pool cannot satisfy every constraint at once (e.g. too few hard questions remain). A **two-phase solve** handles it:

- 1 Try the full model (chapter + difficulty + disjoint + duplicate).
- 2 If infeasible, **drop the difficulty equalities** and re-solve — keeping chapter counts, version disjointness, and duplicate exclusions.

Deliberate asymmetry

Chapter coverage is the **core content requirement** and is never silently violated. Difficulty balance is the **safer constraint to relax** — the run records `difficulty_relaxed` in its diagnostics.

Feature	Greedy	MIP (default)
Decision style	incremental, per version/chapter	whole problem in one model
Chapter counts	exact	exact
Difficulty counts	best effort	exact unless relaxed
Overlap / duplicates	avoided by blocking	mathematically forbidden
Quality	not globally optimised	objective maximises total quality
Best for	fast drafts, large pools	final assembly, tight pools, audit

One interface, two selectors — same constraints, different guarantees.

Deduplication (two levels)

- **Pool-level** (`-dedup`): drop the later member of each near-duplicate pair before assembly.
- **Selector-level**: forbid surviving pairs D during version construction — a second line of defence.

Validation (`-validate`)

A structured report over four dimensions:

- 1 **Schema** validity (round-trips the model)
- 2 **Blueprint** compliance (counts match *exactly*)
- 3 **Grounding** coverage (`source_ref` / overlap)
- 4 **Duplicate** rate

Gate passed = no invalid items *and* blueprint satisfied.

One entry point: `__main__.py` — pool + blueprint/flags → one Quiz JSON per version (each embedding the resolved blueprint).

```
# Pure command line: 20 questions, 10/6/4 across chapters,  
# 40/40/20 difficulty, two disjoint versions, optimal (MIP):  
python -m quizgen --pool questions.json \  
    --total 20 --chapters 1:10,2:6,3:4 \  
    --difficulty 0.4,0.4,0.2 --versions 2 --selector mip  
  
# Blueprint file + dedup + per-version validation report:  
python -m quizgen --pool questions.json --blueprint blueprint.yaml \  
    --dedup --validate --out-dir out
```

Flags **override** the YAML file. Exits non-zero if any version is non-compliant, so it composes in scripts and CI.

In code	Technique	Source lineage
assemble.py MIP	ATA as a 0–1 MIP	van der Linden, <i>Linear Models for Optimal Test Design</i> (2005)
quality objective	objective-weighted selection	van der Linden (2005); here a quality proxy
blueprint.py	table of specifications	classical assessment design
bloom_level	revised Bloom taxonomy	Anderson & Krathwohl (2001); Bloom (1956)
embedding dedup	semantic near-dup detection	Reimers & Gurevych (2019), S-BERT
<i>Project engineering</i> : largest-remainder apportionment, greedy baseline, Jaccard fallback, YAML+flag override precedence.		

- **quizgen implements the *selection* half** of automated exam construction as an **exact optimisation problem**.
- Given a tagged question bank + a blueprint, it assembles **blueprint-compliant, mutually disjoint** versions via a **0–1 MIP**, with a greedy baseline behind the same interface.
- An optional **dedup** pass and a layered **validation** report (schema, blueprint, grounding, duplicates) verify the result.
- **No LLM, no document processing** — fully offline; constraints via a blueprint file and/or CLI flags.

Thank you — questions?

Jiniku Hayashi · KWARC · FAU Erlangen-Nürnberg